

IT Expense Installation And VM Handover Guide

Installation, verification, and VM copy guide for internal handover.

This document explains how to install, run, verify, and copy the IT Expense project on a Windows VM or internal server.

Use this guide when preparing a new machine, handing the project to IT, or cloning a working VM for backup/UAT/internal deployment.

1. Project Summary

Project name: IT Expense

Purpose: Internal IT expense and procurement workflow system.

Main parts:

- Angular frontend
- Express.js backend API
- SQL Server database
- Local file upload folder for attachments

Main project folder:

```
C:\ITExpense\app
```

Recommended server folders:

```
C:\ITExpense
  app
    backend
    dist
    package.json
    package-lock.json
    .env
  uploads
  logs
```

Current API port:

```
5000
```

Development frontend port:

```
4200
```

Production frontend:

```
http://server-name:5000
```

API health check:

```
http://server-name:5000/api/health
```

2. Required Software

Install these on the VM/server:

- Windows Server or Windows 10/11 for internal testing.
- Node.js compatible with this project. Use Node 24 LTS if available, or a compatible Node 22+ runtime.
- SQL Server 2022 or newer, or SQL Server Express for a small internal setup.
- SQL Server Management Studio or Azure Data Studio.

- Git, if the project will be pulled from source control.
- Optional for production hosting: IIS with URL Rewrite and Application Request Routing.
- Optional for running the API as a service: PM2, NSSM, or another Windows service wrapper.

Check installed versions:

```
node -v
npm -v
```

3. Copy Or Download The Project

Place the project in the app folder:

```
C:\ITExpense\app
```

If using Git:

```
cd C:\ITExpense
git clone <repository-url> app
cd C:\ITExpense\app
```

If copying from another machine, copy the project folder but do not rely on copied node_modules. Install dependencies again on the target VM.

Install dependencies:

```
cd C:\ITExpense\app
npm ci
```

If npm ci fails because package-lock.json is missing or changed, use:

```
npm install
```

4. Create The Database

Open SQL Server Management Studio as administrator.

Run these scripts in order:

```
backend/db/01-create-database.sql
backend/db/02-schema.sql
backend/db/03-seed-reference-data.sql
```

For an existing database that was created before the latest SQL integration, also run:

```
backend/db/04-migrate-existing-database.sql
```

For UAT/demo testing only, optionally run:

```
backend/db/06-seed-demo-users.sql
```

Before real production use, remove demo data:

```
backend/db/07-remove-demo-data.sql
```

The database created by the scripts is:

```
ITExpense
```

The database schema is:

```
app
```

The default SQL login used by the app is:

```
it_expense_app
```

Important: change the placeholder password in backend/db/01-create-database.sql before production use.

5. Configure Environment File

Copy the example file:

```
cd C:\ITExpense\app
Copy-Item .env.example .env
```

Edit .env for the VM/server:

```
NODE_ENV=production
PORT=5000
CLIENT_ORIGIN=http://your-internal-server-name

DATA_STORE=sqlserver
DB_SERVER=your-sql-server
DB_PORT=1433
DB_NAME=ITExpense
DB_USER=it_expense_app
DB_PASSWORD=your-real-password
DB_ENCRYPT=true
DB_TRUST_SERVER_CERTIFICATE=false

JWT_SECRET=your-long-random-secret
JWT_EXPIRES_IN=1h
REFRESH_TOKEN_DAYS=7
ALLOW_DEV_PASSWORDS=false

UPLOAD_DIR=C:\ITExpense\uploads
MAX_UPLOAD_MB=5
```

For local SQL Express, the database server value may look like this:

```
DB_SERVER=SERVER-NAME\SQLEXPRESS
DB_ENCRYPT=false
DB_TRUST_SERVER_CERTIFICATE=true
```

Do not commit .env to source control. It contains passwords and secrets.

6. Create Upload Folder

Create the upload folder:

```
New-Item -ItemType Directory -Force C:\ITExpense\uploads
```

Give the Windows account running the Node API read/write access to:

```
C:\ITExpense\uploads
```

7. Build And Verify The Project

From the project folder:

```
cd C:\ITExpense\app
```

Check backend JavaScript syntax:

```
npm run check:backend
```

Check SQL Server connection:

```
npm run check:sql
```

Build the Angular frontend:

```
npm run build
```

The frontend build output is:

```
dist/it-expense/browser
```

8. Run The App

Start the API and production frontend from Node:

```
npm run start:api
```

When the Angular build exists, the backend also serves the frontend.

Open:

```
http://server-name:5000
```

Check the API:

```
http://server-name:5000/api/health
```

For local development, run the API and Angular dev server separately:

```
npm run start:api  
npm start
```

Development frontend:

```
http://localhost:4200
```

Note: `proxy.conf.json` controls where the Angular dev server sends `/api` requests. Update it if the API VM/IP changes.

9. Optional IIS Setup

For a cleaner internal URL, IIS can host the Angular build and reverse proxy API calls.

High-level steps:

1. Create an IIS site that points to `C:\ITExpense\app\dist\it-expense\browser`.
2. Install IIS URL Rewrite and Application Request Routing.
3. Enable reverse proxy in Application Request Routing.
4. Add a rewrite rule:

```
Match URL: ^api/(.*)  
Rewrite URL: http://localhost:5000/api/{R:1}
```

5. Bind the site to the internal DNS name.
6. Use HTTPS if the app will be accessed by real users.

The Angular production environment already uses:

```
/api
```

So the frontend and backend can share the same internal host.

10. Run The API As A Service

Do not depend on an open terminal for production or UAT.

One option is PM2:

```
npm install -g pm2  
pm2 start backend/server.js --name it-expense-api  
pm2 save  
pm2 startup
```

Alternative: use NSSM or a Windows service wrapper to run:

```
node C:\ITExpense\app\backend\server.js
```

Service account requirements:

- Read access to `C:\ITExpense\app`
- Read/write access to `C:\ITExpense\uploads`
- Access to SQL Server
- No unnecessary administrator permissions

11. Demo Login Accounts

Demo accounts only work if backend/db/06-seed-demo-users.sql has been run and development passwords are allowed for UAT.

Password:

```
password123
```

Accounts:

```
requester@test.com
endorser@test.com
approver@test.com
finance@test.com
admin@test.com
super@test.com
```

For production:

- Remove demo users with backend/db/07-remove-demo-data.sql.
- Set ALLOW_DEV_PASSWORDS=false.
- Create real users using bcrypt hashes.

Generate a password hash:

```
npm run hash:password -- "StrongTemporaryPassword"
```

Then use:

```
backend/db/08-create-internal-user-template.sql
```

12. VM Copy Or Clone Procedure

Use this section when making a copy of a working VM.

Before copying the VM:

1. Confirm the app runs successfully.
2. Confirm `http://localhost:5000/api/health` returns a healthy response.
3. Run `npm run check:sql`.
4. Stop the Node API service.
5. Back up the SQL Server database.
6. Confirm whether the copy is for production, UAT, backup, or developer use.
7. Record these values:

```
Server name:
Server IP:
SQL Server name:
Database name:
App folder:
Upload folder:
API port:
Windows service name:
Internal URL:
```

After copying the VM:

1. Change the Windows computer name if required.
2. Change the static IP or DNS record if required.
3. Update `.env` if the server name, database server, upload path, or internal URL changed.
4. Update `CLIENT_ORIGIN` in `.env`.
5. Confirm `DB_SERVER` points to the correct SQL Server.
6. Confirm `UPLOAD_DIR` exists and permissions still work.

7. Start the Node API service.
8. Open `http://localhost:5000/api/health`.
9. Open the app URL in a browser.
10. Login with a test or real account.
11. Create one test request and confirm it appears in the dashboard/list.

Important VM copy notes:

- If production and UAT are both online, do not let them point to the same database unless that is intentional.
- Do not reuse a production database for testing unless it is a controlled backup copy.
- Do not keep demo accounts enabled in production.
- If the VM receives a new hostname or IP address, update DNS, firewall rules, IIS bindings, and `.env`.
- Keep `.env` secure because it contains the database password and JWT secret.

13. Firewall And Network Checks

Allow inbound access to the app port if users access Node directly:

```
TCP 5000
```

If using IIS:

```
TCP 80  
TCP 443
```

If SQL Server is on another server, allow:

```
TCP 1433
```

From the app server, test SQL connection:

```
npm run check:sql
```

14. Final Handover Checklist

Use this checklist before handing the VM to your boss or IT.

- Required software installed.
- Project copied to `C:\ITExpense\app`.
- Dependencies installed with `npm ci`.
- SQL Server database created.
- `.env` configured with real server values.
- Upload folder created.
- Upload folder permissions tested.
- `npm run check:backend` passed.
- `npm run check:sql` passed.
- `npm run build` passed.
- `http://localhost:5000/api/health` works.
- App opens in browser.
- Login tested.
- Request workflow tested.

- API runs as a Windows service or PM2 process.
- Firewall rules confirmed.
- VM backup or clone completed.
- Demo users removed before production.
- Database backup plan confirmed.
- .env stored securely.

15. Troubleshooting

Problem: App opens but API calls fail.

Check:

- Node API is running.
- /api/health works.
- IIS reverse proxy or Angular proxy points to the right API server.
- CLIENT_ORIGIN is correct.

Problem: SQL connection fails.

Check:

- DB_SERVER, DB_NAME, DB_USER, and DB_PASSWORD in .env.
- SQL Server allows TCP/IP.
- SQL login is enabled.
- Firewall allows SQL Server traffic.
- The database scripts ran successfully.

Problem: Login fails in UAT.

Check:

- Demo users were seeded if using demo accounts.
- ALLOW_DEV_PASSWORDS=true only for UAT demo accounts.
- For real users, password hashes were generated and inserted correctly.

Problem: File upload fails.

Check:

- UPLOAD_DIR exists.
- Node service account has write permission.
- File size is under MAX_UPLOAD_MB.

Problem: Production page refresh returns 404.

Check:

- If using Node only, dist/it-expense/browser/index.html exists.
- If using IIS, URL Rewrite sends unknown frontend routes to index.html.

16. Key Files

Important setup files:

package.json

```
.env.example
proxy.conf.json
backend/server.js
backend/db/README.md
backend/db/01-create-database.sql
backend/db/02-schema.sql
backend/db/03-seed-reference-data.sql
backend/db/06-seed-demo-users.sql
backend/db/07-remove-demo-data.sql
backend/db/08-create-internal-user-template.sql
docs/deployment/sql-server-and-server-setup.md
```

Main commands:

```
npm ci
npm run check:backend
npm run check:sql
npm run build
npm run start:api
```